Karthik Reddy Basupally

+91 9063007185 - basupallykarthikreddy@gmail.com - linkedin - github - portfolio

EDUCATION

VIT University **Vellore, Tamil Nadu, India**
B.Tech in Electronics & Communication Engineering, CGPA-8.71/10.0 *Sep 2020 — May 2024*

TECHNICAL SKILLS

Programming Languages: Python, Java (8–21), SQL

Agentic Systems: Multi-Agent Orchestration, Agent Routing & Hand-offs, Hierarchical Delegation, Agent Hooks, Skills & Custom Agents, MCP, Tool/Function Calling, Agent Memory & State Management, Guardrails, Context Engineering, GitHub Copilot Agents

AI / LLM Engineering: LangGraph, LangChain, AWS Bedrock, LLMs, RAG, LLM Evaluation, LLM Observability, Prompt Engineering & Templating, Conversational Memory, Context Ranking

Retrieval & Data: Vector Databases (PGVector, Weaviate), Embedding Models, Sentence Transformers, Semantic Search, Document Parsing & Chunking Strategies, Ingestion Pipelines

Infrastructure & Observability: Docker, GitLab CI/CD, Grafana, MS SQL Server, PostgreSQL, Git, Jira/Confluence

WORK EXPERIENCE

Lead Software Engineer – Agentic AI
Persistent Systems, India *Sep 2026 — Present*

- Architected an **Agentic Orchestration Harness** on **GitHub Copilot custom agents**: one user-facing **orchestrator** that breaks work into bounded tasks and delegates them to division- and team-level **specialist sub-agents** in a **hierarchical multi-agent** tree up to five levels deep.
- Designed **agent routing and delegation**: each caller holds its own allow-list of child agents, path-scoped instructions route domain context to the right agent, and **hand-offs** manage user-directed transitions; a structured **status contract** lets the orchestrator aggregate results and escalate deterministically.
- Built **guardrails and governance**: **least-privilege tool scoping**, **human-in-the-loop** approval gates, scoped edits, lifecycle **hooks**, and optional **MCP** tool integration and agent memory.
- Made orchestration config-driven and portable with YAML **workflow definitions**, JSON schemas, reusable **skills**, and validators for agent discovery and workflow references; separated run state, logs, and memory from source so the harness ships inside any repository.

Senior Software Engineer
Nomura, Mumbai, India *Jul 2024 — Aug 2026*

- Developed **Agent Mode**, a **multi-agent system** on **LangGraph/LangChain** with agentic routing, stateful graphs, agent hand-offs, and **MCP** tool-calling across Confluence, Jira, release management, and financial data platforms; served it through async **FastAPI** services with streaming, batching, and caching to cut latency and inference cost.
- Built a hybrid **RAG** retrieval pipeline for **10,000+ indexed documentation pages**, combining **BM25 sparse retrieval** with dense vector search on **PGVector**, merged via **Reciprocal Rank Fusion (RRF)** and re-ranked using a **cross-encoder** to improve retrieval of both exact-match terms and semantic queries for **15,000+ users**.
- Designed and shipped the enterprise **RAG platform** with personal knowledge bases and organization-wide shared vector stores, building the end-to-end ingestion pipeline for parsing, chunking, embeddings, metadata, indexing, and temporary/persistent stores.
- Built a **RAG evaluation pipeline** measuring **faithfulness, answer relevancy, contextual precision, contextual recall**, and a composite RAG score, enabling retrieval, chunking, and prompt changes to be evaluated against baselines rather than intuition.

PROJECTS

- **SonarAutoFix – LLM-Powered CI/CD Remediation (Nomura)** - Conceptualized and built an agentic AI remediation platform using Dockerized GitLab Runners, LLM workflows, and Javalang parsers to analyze SonarQube findings, generate code fixes, and raise Merge Requests for developer review; remediated **40%** of SonarQube issues across enterprise microservices, earned the **Nomura Star / Quarterly Recognition Award**, and led directly to my transfer to the **AI Center of Excellence**.
- **CodeRAG – Code-Aware Retrieval-Augmented Generation (RAG)** - Developed a code-aware RAG system that models source code using method signatures, call graphs, static analysis, and vector embeddings instead of flat-text chunking, enabling semantic code search and conversational querying through a FastAPI backend using Python, Sentence Transformers, ChromaDB/PGVector/Weaviate, React, and TypeScript. GitHub